ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 12

Formularios, archivos y ficheros

- Repaso rápido: $_GET y $_POST
- Dos páginas: una pregunta y otra responde
- Todo en la misma página
- ¿Una página o dos? Lo que gana y lo que pierde cada una
- Separar la lógica de la vista: la semilla del MVC
- El patrón del formulario profesional
- Validar: comprobar que el dato es correcto
- Escapar: htmlspecialchars y los ataques XSS
- Guardar datos en un fichero JSON
- Post/Redirect/Get: adiós al «¿Desea reenviar el formulario?»
- Subir archivos
- Para saber más: ficheros CSV



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Hasta ahora éramos nosotros quienes escribíamos los datos en el código. Pero una web de verdad es **interactiva**: los usuarios escriben su nombre, eligen opciones, suben fotos y pulsan botones. Hoy aprendemos a **recibir** todo eso, a **comprobarlo** y a **guardarlo** en ficheros.

Ya hicisteis un formulario en la clase 6. Hoy lo convertimos en uno «profesional»: que muestre los errores junto a cada campo, que no borre lo que el usuario ya había escrito y que no se deje colar código malicioso.

Os adelanto la frase de la clase, la que os voy a repetir hasta que la soñéis: **nunca os fiéis de lo que envía el usuario**.

1 📘 Repaso rápido: $_GET y $_POST



2 👍 Dos páginas: una pregunta y otra responde



3 🚀 Todo en la misma página



4 🤔 ¿Una página o dos? Lo que gana y lo que pierde cada una



5 😮 Separar la lógica de la vista: la semilla del MVC



6 👌 El patrón del formulario profesional



7 🤔 Validar: comprobar que el dato es correcto



8 🐞 Escapar: htmlspecialchars y los ataques XSS



9 👍 Guardar datos en un fichero JSON



10 😮 Post/Redirect/Get: adiós al «¿Desea reenviar el formulario?»



11 🚀 Subir archivos



12 🔍 Para saber más: ficheros CSV



13 🚀 Ejercicios



14 👋 Lo que tenéis que llevaros de hoy





CLASE 12 · PUNTO 1

📘 Repaso rápido: $_GET y $_POST

PHP guarda lo que llega del navegador en unos arrays especiales, las **superglobales**, disponibles en cualquier parte del código [1]:

	`$_GET`	`$_POST`
Dónde viajan los datos	En la URL, tras el `?`: `buscar.php?q=zapatos`	Ocultos en el cuerpo de la petición
Se pueden guardar en favoritos o compartir	✅ Sí	❌ No
Úsalo para...	Búsquedas, filtros, número de página	Registros, logins, altas y cualquier cosa que **cambie** datos



⚠️ **Las contraseñas, siempre por POST.** Por GET acabarían en la barra de direcciones, en el historial del navegador y a la vista de cualquiera que mire la pantalla.



CLASE 12 · PUNTO 2

👍 Dos páginas: una pregunta y otra responde

Empecemos por la forma más fácil de entender, la que usaba la web en sus primeros tiempos: **dos archivos**. Uno **pregunta** (el formulario) y otro **responde** (recibe los datos y hace algo con ellos). ¿Recordáis el restaurante de la clase 1? El formulario es la carta con la comanda; el camarero la lleva a la cocina, que es la página que recibe.

Esquema

```
01-formulario.html                          02-recibir.php

┌──────────────────────┐          POST      ┌──────────────────────┐
│ Nombre: [ Ana      ] │ ─────────────────▶ │ $_POST["nombre"] → "Ana"
│ Email:  [ ana@...  ] │                    │ $_POST["email"]  → "ana@…"
│      [ Inscribirme ] │                    │ valida y responde
└──────────────────────┘                    └──────────────────────┘
```

La página que **pregunta** es HTML puro. Lo importante son dos atributos del `<form>` (`01-formulario.html`):

plantilla.php — php-paso-a-paso/clase12 — Visual Studio Code

plantilla.php

```php
<!-- action: QUIÉN recibe los datos. method: CÓMO viajan -->
<form action="02-recibir.php" method="post">
    <p>
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" name="nombre">
    </p>
    <p>
        <label for="email">Email</label>
        <input type="text" id="email" name="email">
    </p>
    <button type="submit">Inscribirme</button>
</form>
```

- `action` es la dirección de la página que recibirá los datos.
- `method` dice cómo viajan: `get` (en la URL) o `post` (ocultos).
- El `name` de cada campo será la **clave** con la que lo leeremos en PHP.

Y la página que **responde** (`02-recibir.php`):

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

// Esta página SOLO recibe: los datos llegan en $_POST con la clave del name de cada campo
$errores = [];

$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($nombre === "") {
    $errores[] = "El nombre es obligatorio.";
}
if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
    $errores[] = "El email no tiene un formato válido.";
}
?>
<?php if (count($errores) === 0): ?>
    <h1>¡Inscripción recibida!</h1>
    <p>Gracias, <?= htmlspecialchars($nombre) ?>. Te escribiremos a <?=
htmlspecialchars($email) ?>.</p>
<?php else: ?>
    <h1>Faltan datos</h1>
    <ul>
        <?php foreach ($errores as $error): ?>
            <li><?= $error ?></li>
        <?php endforeach; ?>
    </ul>
    <!— El usuario tiene que volver atrás… y puede que lo escrito se haya perdido —>
    <p><a href="01-formulario.html">← Volver al formulario</a></p>
<?php endif; ?>
```

Abrid `01-formulario.html`, rellenadlo y enviadlo. Fijaos en la barra de direcciones: **habéis cambiado de página**, de `01-formulario.html` a `02-recibir.php`.

Funciona, y es muy fácil de entender: cada archivo hace **una sola cosa**. Pero haced estas tres pruebas:

Prueba	Qué pasa	¿Problema?
Enviad el formulario vacío	Sale la lista de errores en **otra** página	El usuario tiene que pulsar «Volver» y buscar qué estaba mal
Escribid el nombre con un email mal escrito y volved	Según el navegador, el nombre sigue escrito… o no	No podemos rellenar los campos desde PHP: el formulario es HTML puro
Abrid `02-recibir.php` directamente, sin pasar por el formulario	Salen los errores aunque nadie ha enviado nada	La página que recibe no sabe si la han llamado desde el formulario



CLASE 12 · PUNTO 3

🚀 Todo en la misma página

La solución a esos tres problemas es que **la misma página** pregunte **y** responda. El formulario se envía a sí mismo: basta con **no poner** `action` (o dejarlo vacío). Pero entonces la página necesita saber en cuál de las dos situaciones está:

- Si la hemos **abierto** escribiendo la dirección o con un enlace, llega por **GET**: solo hay que enseñar el formulario.
- Si venimos de **pulsar el botón**, llega por **POST**: hay que validar y responder.

Eso nos lo dice `$_SERVER["REQUEST_METHOD"]` (`03-una-sola-pagina.php`):

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

$errores = [];
$inscrito = false;

// ¿Venimos de pulsar el botón (POST) o solo estamos abriendo la página (GET)?
$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($esEnvio) {
    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errores[] = "El email no tiene un formato válido.";
    }
    $inscrito = count($errores) === 0;
}
?>
<?php if ($inscrito): ?>
    <p>Gracias, <?= htmlspecialchars($nombre) ?>. Te escribiremos a <?=
htmlspecialchars($email) ?>.</p>
<?php else: ?>
    <?php foreach ($errores as $error): ?>
        <p style="color: #b00020"><?= $error ?></p>
    <?php endforeach; ?>

    <!-- Sin action: el formulario se envía a esta misma página -->
    <form method="post">
        <input type="text" id="nombre" name="nombre" value="<?= htmlspecialchars($nombre) ?
>">
        <input type="text" id="email" name="email" value="<?= htmlspecialchars($email) ?>">
        <button type="submit">Inscribirme</button>
    </form>
<?php endif; ?>
```

Repetid las tres pruebas de antes:

- Los errores salen **encima del formulario**, sin cambiar de página.
- Lo que el usuario escribió **sigue en los campos**, porque ahora el formulario es PHP y lo rellenamos con `value="..."`.
- Si se abre la página sin enviar nada, `$esEnvio` es `false` y no sale ningún error: solo el formulario vacío.



CLASE 12 · PUNTO 4

🤔 ¿Una página o dos? Lo que gana y lo que pierde cada una

	Dos páginas	Una sola página
Errores	En otra página; el usuario tiene que volver	Junto al formulario, en el sitio
Datos escritos	Se pueden perder al volver	Se conservan siempre (`value`)
Abrir la página «a pelo»	La que recibe muestra errores falsos	Distingue GET de POST
Archivos que mantener	Dos, y sus `name` tienen que coincidir	Uno
Lo fácil de leer	Cada archivo hace **una** cosa	El archivo hace **dos**: decidir y pintar

Para el **usuario**, gana claramente la página única, y por eso es la que usaremos a partir de ahora. Pero fijaos en la última fila: hemos juntado en un solo archivo dos trabajos muy distintos:

1. **Decidir**: leer `$_POST`, validar, guardar, elegir qué mensaje se muestra. Es **lógica**.
2. **Pintar**: el HTML, con sus etiquetas y sus estilos. Es **presentación**.

Con un formulario de dos campos no pasa nada. Pero con veinte campos, una consulta a la base de datos y una página con mucho diseño, ese archivo se convierte en una sopa de PHP y HTML donde nadie encuentra nada. Y si en vuestro equipo hay alguien que solo sabe de diseño, tendrá que pelearse con vuestro código para cambiar un color. 😅



🎯 **¿Entonces qué es mejor?** Ni una cosa ni la otra tal cual: **una sola dirección para el usuario, pero el trabajo repartido en dos archivos para nosotros**. Lo vemos ahora mismo.



CLASE 12 · PUNTO 5

😮 Separar la lógica de la vista: la semilla del MVC

Vamos a coger la página única y partirla en dos archivos, cada uno con **un solo trabajo** (`04-separado/`):

Esquema

```
04-separado/
├── inscripcion.php              ← el que DECIDE: lee, valida y elige qué mostrar
└── vistas/
    └── inscripcion.vista.php    ← el que PINTA: solo HTML con huecos para los datos
```

El primero, el que decide, no escribe **ni una etiqueta HTML** (`inscripcion.php`):

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
declare(strict_types=1);

// EL CONTROLADOR: recibe la petición, comprueba los datos y decide qué se muestra.
// No escribe ni una etiqueta HTML.

function validarInscripcion(string $nombre, string $email): array
{
    $errores = [];
    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errores[] = "El email no tiene un formato válido.";
    }
    return $errores;
}

$errores = [];
$inscrito = false;

$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($esEnvio) {
    $errores = validarInscripcion($nombre, $email);
    $inscrito = count($errores) === 0;
    // Aquí iría guardar la inscripción en un fichero o en la base de datos
}

// LA VISTA: solo pinta con los datos que le hemos preparado
require __DIR__ . "/vistas/inscripcion.vista.php";
```

Y el segundo, la **vista**, es el HTML de antes, sin una sola decisión «de negocio»: no lee `$_POST` , no valida, no guarda. Solo usa las variables que le ha dejado preparadas el primero: `$inscrito` , `$errores` , `$nombre` y `$email` (`inscripcion.vista.php`).

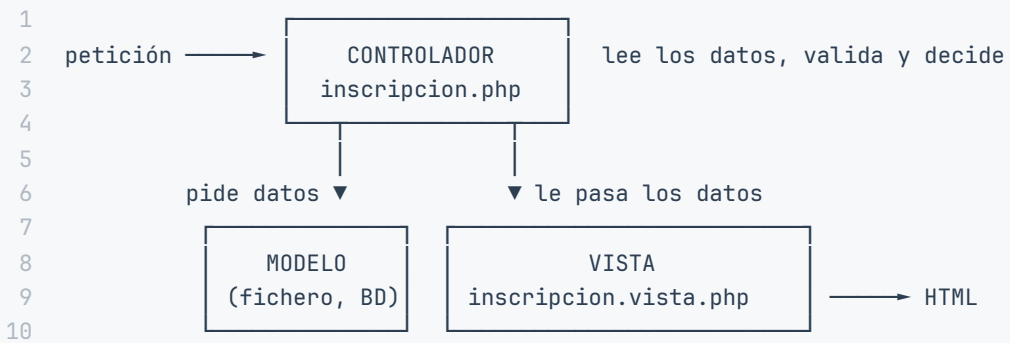
Para el usuario no ha cambiado nada: sigue habiendo **una sola dirección**, `inscripcion.php` , que pregunta y responde. Pero para nosotros todo está en su sitio:

¿Quiero cambiar…?	Abro…
Una regla de validación	`inscripcion.php`
El diseño, un texto o un color	`vistas/inscripcion.vista.php`
Dónde se guardan los datos	`inscripcion.php` (y, más adelante, otro archivo solo para eso)

Esto tiene nombre: MVC

Lo que acabáis de hacer es la idea en la que se basan **todos** los *frameworks* modernos, Laravel incluido. Se llama **MVC** (*Modelo, Vista, Controlador*) y la construiremos a lo grande a partir de la clase 23:

Esquema



- El **controlador** es el jefe de cocina del restaurante: recibe la comanda y organiza.
- La **vista** es el emplatado: cómo se presenta el plato.
- El **modelo**, que todavía no tenemos, será la despensa: guardar y leer los datos (en la clase 15, de la base de datos).



💡 No os agobiéis con los nombres: de momento basta con una costumbre. **Arriba, la lógica; abajo, la vista**, y cuando la página crezca, la vista a su propio archivo. Es exactamente el orden de programa que seguimos desde la clase 10: entradas y procesamiento primero, salidas al final.



CLASE 12 · PUNTO 6

👌 El patrón del formulario profesional

Ahora que sabéis por qué, vamos con un formulario de contacto completo que se procesa **en el mismo archivo**, con la lógica arriba y la vista abajo. Fijaos en que sigue el orden de programa que vimos en la clase 10, porque lo usaréis siempre:

Parte	En un formulario
Funciones	`e()` para escapar y `validarContacto()`, que devuelve un array con los errores
Inicialización	`$errores = [];` y `$enviado = false;`
Entradas	¿Me han enviado el formulario? (`$_SERVER["REQUEST_METHOD"]`) y los datos, limpios con `trim()`
Procesamiento	Si es un envío: validar y, si no hay errores, procesar (guardar, enviar...)
Salidas	El HTML: los errores y el formulario con lo que el usuario ya había escrito

Y en código:

05-formulario-contacto.php — php-paso-a-paso/clase12 — Visual Studio Code

05-formulario-contacto.php

```php
<?php
declare(strict_types=1);

function e(string $texto): string
{
    return htmlspecialchars($texto, ENT_QUOTES, "UTF-8");
}

function validarContacto(string $nombre, string $email): array
{
    $errores = [];
    if ($nombre === "") {
        $errores[] = "El nombre es obligatorio.";
    }
    if (filter_var($email, FILTER_VALIDATE_EMAIL) === false) {
        $errores[] = "El email no tiene un formato válido.";
    }
    return $errores;
}

$errores = [];
$enviado = false;

$esEnvio = $_SERVER["REQUEST_METHOD"] === "POST";
$nombre = trim($_POST["nombre"] ?? "");
$email = trim($_POST["email"] ?? "");

if ($esEnvio) {
    $errores = validarContacto($nombre, $email);
    $enviado = count($errores) === 0;
}
?>
<!— y aquí, el HTML —>
```

En el HTML volvemos a poner en cada campo lo que el usuario escribió, para que no tenga que rellenarlo todo otra vez por un solo error:

05-formulario-contacto.php — php-paso-a-paso/clase12 — Visual Studio Code

05-formulario-contacto.php

```php
<input type="text" name="nombre" value="<?= e($nombre) ?>">
```

¿Y qué hace exactamente esa `e()`? Lo explico en el punto 8.

📄 Ejemplo completo: `05-formulario-contacto.php`



💡 **¿Por qué `REQUEST_METHOD` y no `isset($_POST["enviar"])` ?** Porque si el usuario envía el formulario pulsando **Intro** en lugar del botón, en algunos casos el botón no viaja y vuestro código no se enteraría del envío. Preguntar por el método funciona siempre.

✅ Tres pruebas para comprobarlo

Haced esto	Debéis obtener
Enviad el formulario vacío	Tres errores en rojo
Escribid un nombre y un email mal escrito	Un error del email... **y el nombre sigue escrito**
Rellenadlo bien	El mensaje de agradecimiento



CLASE 12 · PUNTO 7

🤔 Validar: comprobar que el dato es correcto

Validar es comprobar que lo que llega tiene sentido **antes** de usarlo. Y ojo: el atributo `required` del HTML ayuda al usuario, pero **no os protege**. Cualquiera puede saltárselo desde las herramientas del navegador. La validación de verdad se hace **siempre en PHP**.

Qué comprobar	Cómo
Campo obligatorio	`trim($dato) === ""`
Longitud mínima o máxima	`mb_strlen($dato) < 10`
Email	`filter_var($email, FILTER_VALIDATE_EMAIL)` [2]
Número entero en un rango	`filter_var($edad, FILTER_VALIDATE_INT, ["options" => ["min_range" => 16, "max_range" => 99]])`
Valor de una lista (un `<select>`)	`in_array($ciudad, $ciudades, true)`
Dos campos iguales (contraseñas)	`$clave === $clave2`



🎯 **Truco:** si guardáis los errores con el nombre del campo como clave (`$errores["email"] = "..."`), luego podéis mostrar cada error **justo debajo de su campo**. Lo tenéis en la solución del ejercicio 01.



CLASE 12 · PUNTO 8

🐞 Escapar: htmlspecialchars y los ataques XSS

Imaginad un libro de visitas. Alguien escribe esto como «mensaje»:

06-xss.php — php-paso-a-paso/clase12 — Visual Studio Code

🌐 06-xss.php

```
<script>alert('¡Te he hackeado!')</script>
```

Si lo mostráis tal cual con `echo`, el navegador de **cada persona que visite la página** ejecutará ese código. Una alerta es inofensiva, pero en su lugar podría haber un script que robe sesiones. Este ataque se llama **XSS** (*Cross-Site Scripting*) y es de los más comunes en la web [3].

La defensa es sencilla: **escapar** todo lo que venga del usuario antes de mostrarlo. `htmlspecialchars()` convierte los caracteres peligrosos en texto inofensivo [4]:

Carácter	Se convierte en
`<`	`<`
`>`	`>`
`"`	`"`
`'`	`'`

El navegador **muestra** `<script>` en pantalla como texto, pero ya **no lo ejecuta**.

Como lo vais a escribir cientos de veces, mi recomendación es crear una función corta:

06-xss.php — php-paso-a-paso/clase12 — Visual Studio Code

🐘 06-xss.php

```php
<?php
function e(string $texto): string
{
    return htmlspecialchars($texto, ENT_QUOTES, "UTF-8");
}

echo "<p>" . e($comentario) . "</p>";
```

📄 Ejemplo: `06-xss.php`



🛡️ **Las dos reglas de oro: validad al entrar** (¿el dato tiene sentido?) y **escapad al salir** (¿se puede mostrar sin peligro?). Son dos cosas distintas, y hacen falta las dos.



CLASE 12 · PUNTO 9

👍 Guardar datos en un fichero JSON

Hasta que veamos bases de datos, podemos guardar información en **ficheros**. El formato más cómodo es **JSON**: un texto que representa arrays y que entienden PHP, JavaScript y casi cualquier lenguaje [5].

datos.json — php-paso-a-paso/clase12 — Visual Studio Code

datos.json

```
[
    { "nombre": "Ana", "texto": "Hola clase", "fecha": "06/11/2026 17:45" }
]
```

Solo necesitáis cuatro funciones:

Función	Qué hace
`json_encode($array)`	Array → texto JSON
`json_decode($texto, true)`	Texto JSON → array (el `true` es para que devuelva un array asociativo)
`file_put_contents($ruta, $texto)`	Escribe un texto en un fichero (lo crea si no existe)
`file_get_contents($ruta)`	Lee un fichero entero y lo devuelve como texto

07-libro-visitas.php — php-paso-a-paso/clase12 — Visual Studio Code

07-libro-visitas.php

```php
<?php
$archivo = __DIR__ . "/datos/visitas.json";

// Leer
$visitas = [];
if (file_exists($archivo)) {
    $visitas = json_decode(file_get_contents($archivo), true) ?? [];
}

// Añadir y guardar
$visitas[] = ["nombre" => $nombre, "texto" => $texto];
file_put_contents($archivo, json_encode($visitas, JSON_PRETTY_PRINT |
JSON_UNESCAPED_UNICODE), LOCK_EX);
```

Tres detalles de ese código:

- `__DIR__` es la carpeta donde está el archivo PHP. Con `__DIR__ . "/datos/..."` la ruta funciona aunque mováis el proyecto de sitio.
- `JSON_PRETTY_PRINT` y `JSON_UNESCAPED_UNICODE` hacen que el fichero sea legible y que las tildes se vean bien.
- `LOCK_EX` evita que dos personas escriban a la vez y se estropee el fichero.

📄 Ejemplo: `07-libro-visitas.php`. Abrid `datos/visitas.json` después de firmar: es texto normal.



CLASE 12 · PUNTO 10

😮 Post/Redirect/Get: adiós al «¿Desea reenviar el formulario?»

Firmad en el libro de visitas y pulsad **F5**. Si no hacemos nada, el navegador pregunta «¿Desea reenviar el formulario?»… y si decís que sí, la firma se guarda **dos veces**. En una tienda, sería un pedido duplicado. 😬

La solución es un patrón con nombre propio: **Post/Redirect/Get**. Después de procesar el POST con éxito, **redirigimos** a la misma página con una petición GET:

ejemplo.php — php-paso-a-paso/clase12 — Visual Studio Code

ejemplo.php

```php
<?php
// ...guardar los datos...
header("Location: 07-libro-visitas.php?ok=1");
exit;
```

Ahora F5 recarga el GET, que es inofensivo. Y el `?ok=1` nos sirve para mostrar el mensaje de «¡Gracias por firmar!».



⚠️ **Nunca olvidéis el `exit;` después de `header("Location: ...")`.** La cabecera le dice al navegador que se vaya, pero PHP **sigue ejecutando** lo que hay debajo si no lo paráis [6].

⚠️ Y `header()` tiene que ir **antes de cualquier salida**: ni un `echo`, ni HTML, ni un espacio en blanco antes. Si no, veréis el famoso error *«headers already sent»*. Por eso procesamos el formulario arriba del todo y el HTML va después.



CLASE 12 · PUNTO 11

🚀 Subir archivos

Para subir fotos o documentos, el formulario necesita dos cosas: `method="post"` y, **obligatoriamente**, `enctype="multipart/form-data"`. Sin ese atributo, el archivo no viaja.

08-subir-foto.php — php-paso-a-paso/clase12 — Visual Studio Code

🌐 08-subir-foto.php

```
1 <form method="post" enctype="multipart/form-data">
2     <input type="file" name="foto">
3     <button type="submit">Subir</button>
4 </form>
```

Los archivos no llegan a `$_POST`, sino a `$_FILES` [7]:

Clave en `$_FILES["foto"]`	Qué contiene
`["name"]`	El nombre original en el ordenador del usuario. **No lo uséis para guardar**
`["tmp_name"]`	Dónde lo ha dejado PHP temporalmente. Es el archivo de verdad
`["size"]`	Tamaño en bytes
`["error"]`	`UPLOAD_ERR_OK` (vale 0) si todo ha ido bien

Las comprobaciones que no podéis saltaros

08-subir-foto.php — php-paso-a-paso/clase12 — Visual Studio Code

08-subir-foto.php

```php
<?php
$foto = $_FILES["foto"] ?? null;
$tiposPermitidos = ["image/jpeg" => "jpg", "image/png" => "png", "image/webp" => "webp"];

if ($foto === null || $foto["error"] !== UPLOAD_ERR_OK) {
    $mensaje = "No ha llegado ningún archivo.";
} elseif ($foto["size"] > 2 * 1024 * 1024) {
    $mensaje = "Máximo 2 MB.";
} else {
    $info = getimagesize($foto["tmp_name"]);     // ¿es de verdad una imagen?
    $tipo = $info["mime"] ?? "";

    if (!isset($tiposPermitidos[$tipo])) {
        $mensaje = "Solo JPG, PNG o WEBP.";
    } else {
        $nombreFinal = bin2hex(random_bytes(8)) . "." . $tiposPermitidos[$tipo];
        move_uploaded_file($foto["tmp_name"], __DIR__ . "/uploads/" . $nombreFinal);
    }
}
```

¿Por qué tanto lío?

- **Comprobamos el contenido, no el nombre.** Cualquiera puede llamar `foto.jpg` a un programa malicioso. `getimagesize()` mira dentro del archivo.
- **Inventamos un nombre nuevo y aleatorio.** Así nadie puede sobrescribir otro archivo ni colar un nombre peligroso.
- `move_uploaded_file()` mueve el archivo temporal a su sitio definitivo. Si no lo movéis, PHP lo borra al terminar.

📄 Ejemplo: `08-subir-foto.php`



💡 Si una foto grande da error aunque esté por debajo de vuestro límite, revisad `upload_max_filesize` y `post_max_size` en el `php.ini` (en Laragon: **Menú → PHP → php.ini**). Recordad que con `phpinfo()` podéis ver qué valores tenéis.



CLASE 12 · PUNTO 12

🔍 Para saber más: ficheros CSV

Un **CSV** es una tabla en texto: una fila por línea y los valores separados por `;` o `,`. Excel los abre directamente, así que son perfectos para exportar listados.

09-csv.php — php-paso-a-paso/clase12 — Visual Studio Code

09-csv.php

```php
<?php
$f = fopen(__DIR__ . "/datos/alumnos.csv", "w");    // abrir para escribir
fputcsv($f, ["Marta", 22, "Santander"], ";");
fclose($f);                                          // cerrar siempre
```

Para leerlos se usa `fgetcsv()` dentro de un `while`, línea a línea.

📄 Ejemplo: `09-csv.php`



CLASE 12 · PUNTO 13

🚀 Ejercicios

- ☐ **01 · Registro.** Nombre, email, edad (entre 16 y 99), ciudad (un `<select>`) y contraseña repetida dos veces (mínimo 8 caracteres). Cada error debe aparecer **debajo de su campo** y los datos deben conservarse (menos las contraseñas). → `ej01-registro.php`
- ☐ **02 · Encuesta.** «¿Qué lenguaje te gusta más?» con botones de radio. Guardad los votos en un JSON, aplicad Post/Redirect/Get y mostrad los resultados con su porcentaje. → `ej02-encuesta.php`
- ☐ **03 · Galería.** Un formulario para subir fotos a la carpeta `fotos/` y, debajo, todas las fotos subidas. Pista: `glob()` os da la lista de archivos de una carpeta. → `ej03-galeria.php`
- ☐ **04 · Calculadora web.** Dos números y una operación en un `<select>`. Validad que son números (`is_numeric()`) y la división entre cero, conservad lo escrito y mostrad el resultado. → `ej04-calculadora.php`
- ☐ **05 · Conversor de unidades.** Un formulario **GET** que convierta kilómetros y millas, kilos y libras. Comprobad que el resultado se puede compartir copiando la dirección. → `ej05-conversor.php`
- ☐ **06 · Lista de tareas.** Añadir, marcar como hecha y borrar tareas guardadas en un JSON, siempre con Post/Redirect/Get. Pista: un botón puede llevar `name` y `value`, y así un mismo formulario sabe qué botón se pulsó. → `ej06-tareas.php`
- ☐ **07 · Contador de visitas.** Guardad en `datos/visitas.txt` cuántas veces se ha abierto la página y mostradlo. → `ej07-contador-visitas.php`
- ☐ **08 · Envía tu CV.** Subida de un PDF de 2 MB como máximo, comprobando el tipo real con `mime_content_type()` y guardándolo en `cv/` con un nombre nuevo. → `ej08-subir-cv.php`

- ☐ **09 · Descargar en CSV.** Una página que, al abrirla, **descargue** un `alumnos.csv` que se abra bien en Excel. Pistas: las cabeceras `Content-Type` y `Content-Disposition`, y `fopen("php://output", "w")`. → `ej09-exportar-csv.php`
- ☐ **10 · Leer un CSV.** Leed `datos/productos.csv` y mostradlo en una tabla con el importe de cada línea y el total. Cuidado: los precios vienen con coma decimal. → `ej10-importar-csv.php`
- ☐ **11 · Hamburguesa a medida.** Casillas de extras con `name="extras[]"` (llega un array) y el precio total. Ignorad cualquier extra que no exista en vuestra lista. → `ej11-pedido-extras.php`



🏆 **Reto:** en el libro de visitas, probad a firmar con `<b>Hola</b>` como mensaje. ¿Sale en negrita? ¿Por qué no? Ahora quitad la `e()` en esa línea y repetid la prueba. Volved a ponerla en cuanto lo veáis. 😉



CLASE 12 · PUNTO 14

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
GET o POST	GET para consultar, POST para enviar o cambiar datos. Contraseñas, siempre por POST
Dos páginas	`action="recibir.php"` : fácil de entender, pero los errores salen lejos y se pierde lo escrito
Una página	Sin `action` : el formulario se envía a sí mismo y conserva los datos
¿Me han enviado el formulario?	`$_SERVER["REQUEST_METHOD"] === "POST"`
Lógica y vista	Una dirección para el usuario, dos archivos para nosotros: la semilla del MVC
Validar	En PHP, siempre. El `required` del HTML no protege
Escapar	`htmlspecialchars()` (nuestra `e()`) en todo lo que venga del usuario
Conservar datos	`value="<?= e($nombre) ?>"`
JSON	`json_encode` / `json_decode` + `file_put_contents` / `file_get_contents`
PRG	Tras un POST correcto: `header("Location: ...")` y `exit;`
Subidas	`enctype="multipart/form-data"` , `$_FILES` , comprobar tipo y tamaño, nombre aleatorio

📚 Fuentes y documentación oficial

1. Manual de PHP: superglobales
2. Manual de PHP: filtros de validación
3. OWASP: Cross Site Scripting (XSS)
4. Manual de PHP: htmlspecialchars
5. Manual de PHP: json_encode
6. Manual de PHP: header
7. Manual de PHP: subida de ficheros con el método POST